Lecture 8: Regression Analysis - Part 1

James Sears*
AFRE 891/991 FS 26
Michigan State University

*Parts of these slides are adapted from **"Data Science for Economists"** by Grant McDermott and **"Advanced Data Analytics"** by Nick Hagerty.

Table of Contents

1. Prologue

2. Basic Regression Analysis

3. Fast Fixed Effects Regression

4. (Programmatic) Formulas

5. Standard Errors

Prologue

Empirical Analysis

We spent the first half of our course building a **foundational knowledge** of data manipulation in R.

Last lecture, we expanded what we could do by learning how to do things **programmatically**.

While cleaning, wrangling, and visualizing data can be fun, it's generally not the end goal - that's usually some form of **statistical/econometric analysis**.

Over the next weeks we're going to focus on the analysis side, using both **traditional** and **non-traditional** methods.

Empirical Analysis

This Lecture: Regression

- Standard regression: `lm()`
- Fixed effects regression: **fixest** and `feols()`
- Formulas
- Choice of standard errors
- Regression Tables
- Visualizing regression output
- IV Regression
- Diff-in-Diff
  - Staggered Adoption
- Event Study and Sun-Abraham estimator

Next Lecture: Synthetic Control Methods

- Canonical Synthetic Control
- Synthetic Diff-in-Diff
  - Uniform Adoption
  - Staggered Adoption
- Synthetic Control Prediction Intervals
  - Uniform or Staggered Adoption
  - Range of estimands + specifications

Preparation

891

- Read parts of **Causal Inference: The Mixtape by Scott Cunningham**
  - 7.1-7.2, 7.4 Instrumental Variables
  - 8.2: Panel Data Estimation, Fixed Effects, and Assumptions
  - 9.1-9.2: Canonical Difference-in-Differences
  - 9.4.3 Event Study
  - 9.6.1 and 9.6.8: Bacon Decomposition
- **Abadie et al. (2022)**

991

- If no causal inference/applied econometrics background: read the Mixtape sections outlined on the left
- **Abadie et al. (2022)**
- Sections 1, 3, and 4 of **Cattaneo et al. (2025)**

Prologue

Packages we'll use today:

```
if (!require("pacman")) install.packages("pacman")
pacman::p_load(broom, dslabs, fixest, stargazer, tictoc, tidyverse)
```

as well, let's load data necessary to replicate initial results from **Taylor and Druckenmiller AER (2022)**[2]

```
wl_df ← read_csv("data/zip_PANEL_data.csv") %>%
  drop_na(county_fips, state_fips, claims)
```

[2] There is a *lot* of great stuff in this paper and it's all super replicable and well-documented. I highly recommend checking it out if you're curious (only caution is that there are some *big* files in there).

Taylor and Druckenmiller AER (2022)

"Wetlands, Flooding, and the Clean Water Act" explores the benefits of wetlands in mitigating flood damage



FIGURE 2. WETLANDS AND FLOODING SCHEMATIC

- 2020 EPA decision narrowed wetland protections under Clean Water Act
- Wetlands provide a range of critical ecosystem services that benefit society
- Private benefits to landowners is relatively small

Taylor and Druckenmiller AER (2022)

"Wetlands, Flooding, and the Clean Water Act" explores the benefits of wetlands in mitigating flood damage



FIGURE 2. WETLANDS AND FLOODING SCHEMATIC

This paper estimates the **value of wetlands for flood mitigation**

Let's explore this using regression.

Basic Regression Analysis

Basic Regression Analysis

Suppose we want to estimate the following model:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \beta_3 x_{3i} + u_i$$

We can use base R's `lm()` command (**l**inear **m**odels) to run basic regression analysis:

```
lm(y ~ x1 + x2 + x3, data = df)
```

Two arguments:

1. `formula`: the equation to estimate (`y ~ x1 + x2 + x3`)
2. `data source`: the dataframe containing the variables (`df`)

Basic Regression Analysis

Let's run a basic simple linear regression of national flood insurance claims on total wetland area in zipcode i:

```
floodreg_1 ← lm(claims ~ wetland_ha, data = wl_df)
```

This adds an `lm` object to memory containing a bunch of useful info

```
View(floodreg_1)
```

Basic Regression Analysis

Use the generic `summary()` function to get fairly standard regression output:

```
summary(floodreg_1)
```

```
##
## Call:
## lm(formula = claims ~ wetland_ha, data = wl_df)
##
## Residuals:
##       Min         1Q    Median         3Q       Max
##  -1657005   -105263   -101902   -101588 418101306
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1.016e+05  9.049e+03   11.23  < 2e-16 ***
## wetland_ha  5.948e+00  1.918e+00    3.10  0.00193 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3155000 on 131586 degrees of freedom
```

Basic Regression Analysis

To retrieve just the **coefficients table:**

- `coefficients` element of the regression summary object
- Point estimates and standard errors
- t stat and p-value (for $\beta = 0$)
- Stored as a `matrix`

```
reg1_coef ← summary(floodreg_1)$coefficients
reg1_coef
```

```
##                 Estimate  Std. Error   t value     Pr(>|t|)
## (Intercept) 1.015883e+05 9048.768214 11.226756 3.107159e-29
## wetland_ha  5.947641e+00    1.918354  3.100388 1.933083e-03
```

Basic Regression Analysis

Often we'll want to convert the regression output to a **tidy dataframe**.

We can do this using the `tidy()` function from the **broom** package:

```
tidy(floodreg_1, conf.int = TRUE)
```

```
## # A tibble: 2 × 7
##   term         estimate std.error statistic  p.value conf.low conf.high
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
## 1 (Intercept) 101588.     9049.        11.2  3.11e-29 83853.   119324.
## 2 wetland_ha       5.95      1.92       3.10 1.93e- 3     2.19       9.71
```

broom

Note that **broom** has several other useful functions too.

For instance: `glance()` summarizes model metadata (R^2, AIC, etc.) in a data frame:

```
glance(floodreg_1)
```

```
## # A tibble: 1 × 12
##   r.squared adj.r.squared    sigma statistic p.value    df  logLik
AIC    BIC
##       <dbl>         <dbl>    <dbl>     <dbl>   <dbl> <dbl>   <dbl>
<dbl>  <dbl>
## 1 0.0000730     0.0000654 3154992.      9.61 0.00193     1 -2.16e6
4.31e6 4.31e6
## # ℹ 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

Fast Fixed Effects Regression

Fast Fixed Effects Regression

Our first regression suggests that a one hectare increase in total wetland area is associated with a $2-10 increase in zipcode-level NFIP claims.

Q: what's the problem with this regression?

A: okay, there's definitely more than one. Here are several that immediately stand out:

- Not accounting for the panel nature of the data
- Specification of standard errors to account for potential error correlation
- Other zipcode characteristics likely associated with NFIP claims and wetland area

Fast Fixed Effects Regression

Most regressions you'll run will involve some kind of **fixed effects**

- unit, time, or both (or multiple in each dimension, or interactive...)

Fortunately for us, the **fixest** ("fast fixed effects") package exists.

- `feols()` makes estimating models with large number of fixed effects quick and painless
  - Like orders of magnitude faster than alternatives (`lm()`, `reghdfe` in Stata)
- Handles simple regressions, multiple estimation
- Direct computation of alternate standard errors
- Methods for Diff-in-Diff, IV, event studies, and other estimators
  - GLM, (nonlinear Max) Likelihood, Sun-Abraham estimator
- Wide range of tools to make producing tables and visuals easy

Fast Fixed Effects Regression

First, let's replicate the simple model we ran before using `feols()`:

```
floodreg_1b ← feols(claims ~ wetland_ha, data = wl_df)
```

Conveniently, the basic syntax is exactly the same.

Fast Fixed Effects Regression

We now have a `fixest` object, which contains more than twice the elements of `lm()`.

We can view the output table similarly with `summary()`:

```
summary(floodreg_1b)
```

```
## OLS estimation, Dep. Var.: claims
## Observations: 131,588
## Standard-errors: IID
##                 Estimate Std. Error  t value  Pr(>|t|)
## (Intercept) 101588.31268 9048.76821 11.22676 < 2.2e-16 ***
## wetland_ha       5.94764    1.91835  3.10039 0.0019331 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 3,154,968.2   Adj. R2: 6.545e-5
```

Which now yields a dataframe, but we can still tidy things up to get simpler variable names (and drop the statistical significance factor)

Fast Fixed Effects Regression

Access the coefficients table with `coeftable()`

```
coeftable(floodreg_1b)
```

```
##                 Estimate  Std. Error   t value     Pr(>|t|)
## (Intercept) 1.015883e+05 9048.768214 11.226756 3.107159e-29
## wetland_ha  5.947641e+00    1.918354  3.100388 1.933083e-03
## attr(,"vcov_type")
## [1] "IID"
```

Fast Fixed Effects Regression

View the standard errors individually with `se()`

```
se(floodreg_1b)
```

```
## (Intercept)  wetland_ha
## 9048.768214    1.918354
## attr(,"vcov_type")
## [1] "IID"
```

and confidence intervals with `confint()`

```
confint(floodreg_1b, level = 0.95) # get 95% Conf. Int.
```

```
##                    2.5 %       97.5 %
## (Intercept) 83852.889735 1.193237e+05
## wetland_ha      2.187702 9.707581e+00
```

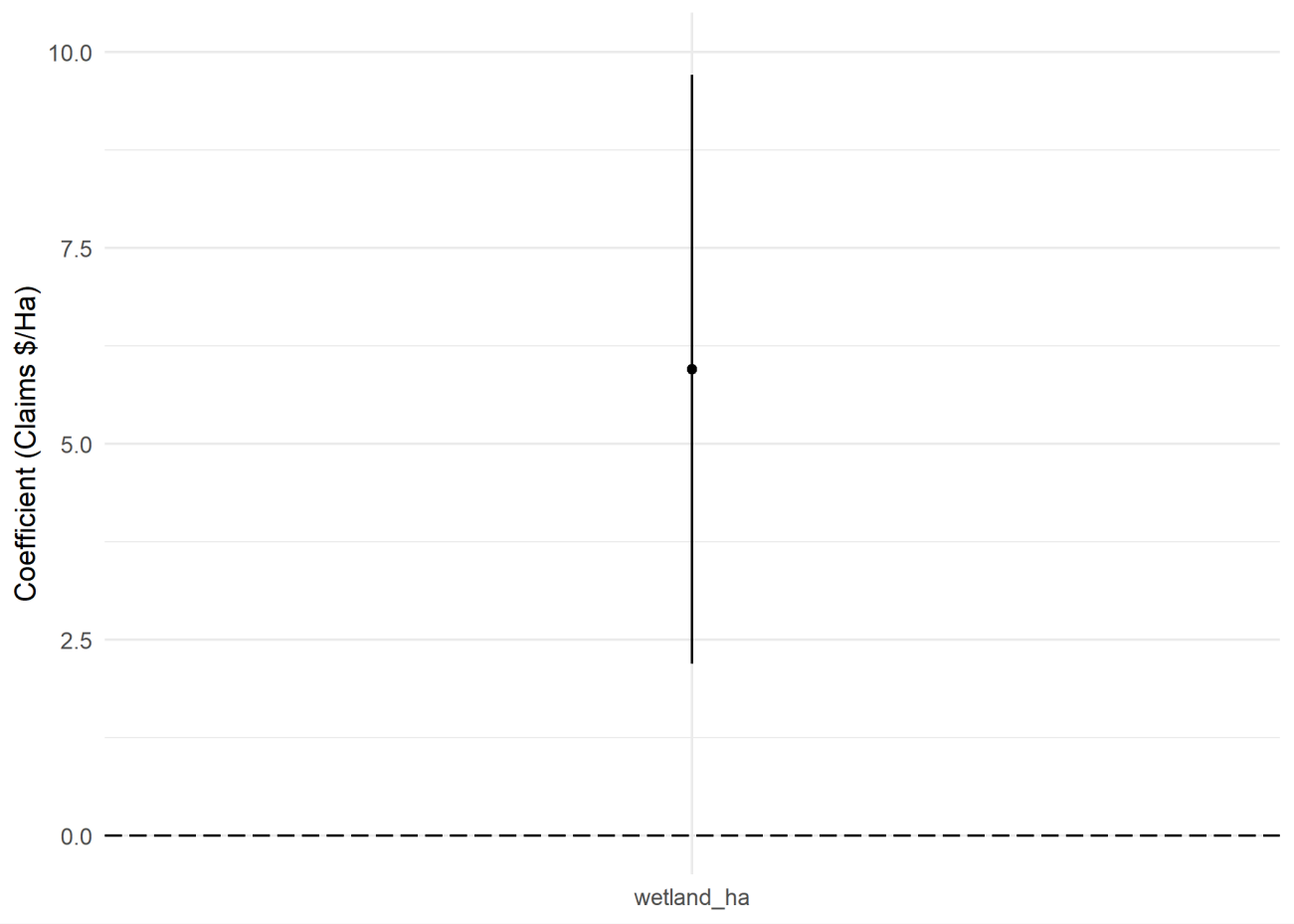
Plotting Regression Coefficients

`tidy()` and `confint()` make it easy to **plot regression output** rather than report in a table.

```
confint_1b ← confint(floodreg_1b, parm = c("wetland_ha"), level = 0.95)
data.frame(var = "wetland_ha",
           estimate = tidy(floodreg_1b)[2, "estimate"],
ci_l = confint_1b[1,1],
ci_u = confint_1b[1,2]
) %>%
  ggplot(aes(x = var)) +
  geom_point(aes(y = estimate)) +
  geom_linerange(aes(ymin = ci_l, ymax = ci_u)) +
  geom_hline(aes(yintercept = 0), linetype = "longdash") +
  lims(y = c(0,10)) +
  labs(y = "Coefficient (Claims $/Ha)", x = NULL) +
  theme_minimal()
```

Plotting Regression Coefficients

`tidy()` and `confint()` make it easy to **plot regression output** rather than report in a table.



Fast Fixed Effects Regression

Easily access fitted values and residuals with `$fitted.values` and `$residuals`, and the input data itself with `fixest_data()`

```
data.frame(
    val = fixest_data(floodreg_1b)$claims[1:10],
    val_hat = floodreg_1b$fitted.values[1:10],
    resid = floodreg_1b$residuals[1:10]
)
```

Adding Fixed Effects

`feols()` makes it easy to add any sort of **fixed effects** you want to your regression:

```
feols(y ~ x | fe_1 + ... + fe_n, data = df)
```

- Add a `|` after your formula and add any fixed effects on the RHS.

For example, if we want to add state fixed effects to our wetlands model:

```
floodreg_2 ← feols(claims ~ wetland_ha | state_fips, data = wl_df)
coeftable(floodreg_2)
```

```
##            Estimate Std. Error   t value    Pr(>|t|)
## wetland_ha -6.34305   2.023058 -3.135378 0.001716706
## attr(,"vcov_type")
## [1] "IID"
```

Viewing Fixed Effects

By default, we don't get the fixed effects estimates presented as coefficients - rather they're netted off in a computationally-efficient fashion prior to estimation (this is why it's so fast).

If you care about the fixed effects coefficients themselves, you can recover them (potentially slowly!) with `fixef()`:

```
fixef(floodreg_2)
```

```
## $state_fips
##            1            2            4            5            6
8
##   57684.8787     745.5667    4892.8401   26938.5931    9804.6145
6686.1870
##            9           10           11           12           13
15
##    2560.6355   45500.1135    4333.5653  221050.0204   34224.6055
26147.3602
##           16           17           18           19           20
```

Adding Fixed Effects

The resulting `fixest` object contains a lot of information on our fixed effects, including

- What variables were used for each fixed effect (`fixef_vars`)
- Coefficient size for each fixed effect dimension (`fixed_size`)

If instead we wanted state and year fixed effects:[3]

```
feols(claims ~ wetland_ha | state_fips + year, data = wl_df) %>%
  coeftable()
```

```
##              Estimate Std. Error   t value    Pr(>|t|)
## wetland_ha -6.353344    2.02262 -3.141146 0.001683255
## attr(,"vcov_type")
## [1] "IID"
```

[3] fixed effects in hifixest functions play nicely with string or numeric variables!

Adding Fixed Effects

Or instead interacted county-year fixed effects:[4]

```
floodreg_int_fe ← feols(claims ~ wetland_ha | county_fips^year, data =
wl_df)
coeftable(floodreg_int_fe)
```

```
##              Estimate Std. Error    t value  Pr(>|t|)
## wetland_ha -0.8240054   2.094844 -0.3933492 0.6940623
## attr(,"vcov_type")
## [1] "IID"
```

```
floodreg_int_fe$fixef_size
```

```
## county_fips^year
##            11764
```

[4] Note that the standard errors have defaulted to clustering at the level of the fixed effects - we'll chat more about this shortly.

Multiple Estimation

`feols()` also makes it easy to run **multiple estimation**.[5]

Using both claims $ and housing values as dependent variables:

```
feols(c(claims, housing_value) ~ wetland_ha | county_fips^year, data =
wl_df) %>%
  summary()
```

[5] See the handy **multiple estimation vignette** for more useful multiple estimation tools (e.g. stepwise inclusion of covariates).

Multiple Estimation

`feols()` also makes it easy to run **multiple estimation**.[5]

Using both claims $ and housing values as dependent variables:

```
feols(c(claims, housing_value) ~ wetland_ha | county_fips^year, data =
wl_df) %>%
  summary()
```

```
##                              x.1                  x.2
## Dependent Var.:           claims         housing_value
##
## wetland_ha       -0.8240 (2.095) -0.6036*** (0.0734)
## Fixed-Effects:   --------------- ------------------
## county_fips-year             Yes                  Yes
## ________________ _______________ __________________
## S.E. type                    IID                  IID
## Observations             130,828              124,232
## R2                       0.41350              0.70751
## Within R2                 1.3e-6              0.00060
## ---
```

Programmatic Formulas

Programmatic Formulas

There are time we might want to avoid the usual syntax and use a **programmatic formula** for our regression.

Suppose we wanted to see the separate effect of woody vs. herbaceous wetland acreage, and include both state and year fixed effects. Instead of typing the formula, we can put it together programmatically:

```
# first, specify the formula components as strings/vectors
depvar ← "claims"

# Use the collapse argument of paste() to combine elements together with
the + separator
x ← c("wetland_herb_ha", "wetland_woody_ha") %>% paste(collapse = " +
")
fe ← c("state_fips", "year")  %>% paste(collapse = " + ")

# Combine it all together into a formula
form ← paste(depvar, "~", x, "|", fe) %>%
  as.formula()
```

Programmatic Formulas

There are time we might want to avoid the usual syntax and use a **programmatic formula** for our regression.

Suppose we wanted to see the separate effect of woody vs. herbaceous wetland acreage, and include both state and year fixed effects. Instead of typing the formula, we can put it together programmatically:

```
form
```

```
## claims ~ wetland_herb_ha + wetland_woody_ha | state_fips + year
## <environment: 0x000002cdcda496c0>
```

Programmatic Formulas

Once we have a formula object, we can use it as the formula object of `feols()` (or `lm()`):

```
# Use the formula as the first argument of feols
feols(form, data = wl_df) %>% summary()
```

```
## OLS estimation, Dep. Var.: claims
## Observations: 131,588
## Fixed-effects: state_fips: 52,  year: 4
## Standard-errors: IID
##                  Estimate Std. Error  t value   Pr(>|t|)
## wetland_herb_ha   18.3309    5.22919  3.50551 4.5589e-04 ***
## wetland_woody_ha -16.1681    2.78686 -5.80155 6.5853e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 3,132,626.1     Adj. R2: 0.013765
##                     Within R2: 2.742e-4
```

Multiple Estimation

Note that multiple estimation may also be an alternative strategy:

```
feols(claims ~ csw(wetland_ha, wetland_herb_ha, wetland_woody_ha) |
county_fips^year, data = wl_df) %>% etable()
```

```
##                              ..1             ..2                  ..3
## Dependent Var.:           claims          claims               claims
##
## wetland_ha       -0.8240 (2.095) -6.479* (3.057) -3.031 (18,882,211.0)
## wetland_herb_ha                   17.48* (6.880)     14 (18,882,211.0)
## wetland_woody_ha                                 -3.438 (18,882,211.0)
## Fixed-Effects:   --------------- --------------- --------------------
## county_fips-year             Yes             Yes                  Yes
## _______________ _______________ _______________ ____________________
## S.E. type                    IID             IID                  IID
## Observations             130,828         130,828              130,828
## R2                       0.41350         0.41353              0.41353
## Within R2                 1.3e-6         5.55e-5              5.55e-5
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Standard Errors

Standard Errors

Let's take a few minutes to chat about the statistical elephant in the room: **standard errors**.

By default, `lm()` uses classical/homoskedastic (IID) standard errors, which you basically should **never** use with observational data.

A distinct advantage of `feols()` is its built-in functionality for **alternate standard errors**.

- Heteroskedasticity-robust ("White")
- Clustered
  - Up to four-way
- Newey-West (heteroskedastic and serially correlated errors)
- Driscoll Kray (cross-sectionally and serially correlated)
- Conley (spatially correlated)

Standard Errors

Many empirical economic studies report **heteroskedasticity-robust** standard errors

- Also known as White or Eicker-Huber-White standard errors

These "Robust" standard errors are **agnostic to the form of error dependency**.

A common alternative is **clustered standard errors**, which are often rationalized through **residual dependency across units in the same cluster**.

- i.e.luster by the geographic unit

Standard Errors

Often these standard errors can differ. By a *lot*.

Let's revisit our first OLS estimate. Recall the **homoskedastic** standard errors:

```
floodreg_iid ← feols(claims ~ wetland_ha, data = wl_df, vcov = "iid")
coeftable(floodreg_iid )
```

```
##                 Estimate  Std. Error   t value     Pr(>|t|)
## (Intercept) 1.015883e+05 9048.768214 11.226756 3.107159e-29
## wetland_ha  5.947641e+00    1.918354  3.100388 1.933083e-03
## attr(,"vcov_type")
## [1] "IID"
```

```
se_iid ← coeftable(floodreg_iid)[2,2] %>% round(4)
```

Standard Errors

If we instead use robust standard errors:

```
floodreg_rob ← feols(claims ~ wetland_ha, data = wl_df,
      vcov = "hetero")
coeftable(floodreg_rob)
```

```
##                 Estimate  Std. Error   t value     Pr(>|t|)
## (Intercept) 1.015883e+05 8975.872203 11.317932 1.104036e-29
## wetland_ha  5.947641e+00    1.563318  3.804499 1.421557e-04
## attr(,"vcov_type")
## [1] "Heteroskedasticity-robust"
```

```
se_rob ← coeftable(floodreg_rob)[2,2] %>% round(4)
```

Standard Errors

Alernatively, we may believe that there is likely correlation in error terms for zip codes within the same **county**

```
floodreg_cl_cty ← feols(claims ~ wetland_ha, data = wl_df,
      cluster = ~ county_fips)
coeftable(floodreg_cl_cty)
```

```
##                 Estimate   Std. Error  t value     Pr(>|t|)
## (Intercept) 1.015883e+05 28861.848410 3.519813 0.0004380073
## wetland_ha  5.947641e+00     2.461991 2.415785 0.0157582439
## attr(,"vcov_type")
## [1] "Clustered (county_fips)"
```

```
se_cl_cty ← coeftable(floodreg_cl_cty)[2,2] %>% round(4)
```

Standard Errors

Or we might be worried that state policies are determining the correlation:

```
floodreg_cl_st ← feols(claims ~ wetland_ha, data = wl_df,
      cluster = ~ state_fips)
coeftable(floodreg_cl_st)
```

```
##                 Estimate   Std. Error t value  Pr(>|t|)
## (Intercept) 1.015883e+05 48232.080206 2.10624 0.0401248
## wetland_ha  5.947641e+00     3.851649 1.54418 0.1287276
## attr(,"vcov_type")
## [1] "Clustered (state_fips)"
```

```
se_cl_st ← coeftable(floodreg_cl_st)[2,2] %>% round(4)
```

Standard Errors

Or that the year also matters:

```
floodreg_cl_styr ← feols(claims ~ wetland_ha, data = wl_df,
      cluster = ~ state_fips+year)
coeftable(floodreg_cl_styr)
```

```
##                 Estimate   Std. Error  t value   Pr(>|t|)
## (Intercept) 1.015883e+05 41784.653908 2.431235 0.09322706
## wetland_ha  5.947641e+00     4.517639 1.316538 0.27953875
## attr(,"vcov_type")
## [1] "Clustered (state_fips & year)"
```

```
se_cl_styr ← coeftable(floodreg_cl_styr)[2,2] %>% round(4)
```

Standard Errors

The choice of standard error **dramatically affects statistical significance.**

Standard Error	Estimate	P-Value
Conventional	1.9184	0.0019
Robust	1.5633	0.0014
Clustered (County)	2.462	0.0158
Clustered (State)	3.8516	0.1287
Clustered (State + Year)	4.5176	0.2795

State-clustered errors are 2.46 times larger than robust ones. So, which do we choose?

Clustered Standard Errors

When it comes to clustering, it turns out that there are three main misconceptions.[6]

1: The need for clustering hinges on the presence of non-zero correlation between residuals for units in the same cluster.

- Presence of non-zero correlation isn't sufficient to justify clustering
- Absence of this correlation doesn't rule out clustering as necessary

[6] I highly recommend reading **"When You Should Adjust Standard Errors for Clustering?" by Abadie et al.** in QJE in 2023 for complete discussion.

Clustered Standard Errors

When it comes to clustering, it turns out that there are three main misconceptions.[6]

2: If clustering standard errors makes a difference, you should do it

- Leads to overly-conservative standard errors in contexts where clustering isn't justifiable
  - i.e. Random sample across clusters but outcomes positively correlated within clusters

[6] I highly recommend reading **"When You Should Adjust Standard Errors for Clustering?" by Abadie et al.** in QJE in 2023 for complete discussion.

Clustered Standard Errors

When it comes to clustering, it turns out that there are three main misconceptions.[6]

3: There are only two choices: fully adjust for clusters, or use robust SEs

- Abadie et al. (2023) show that you can make substantial improvements by combining both

[6] I highly recommend reading **"When You Should Adjust Standard Errors for Clustering?" by Abadie et al.** in QJE in 2023 for complete discussion.

Clustered Standard Errors

In short, it's the **sampling process** and the **treatment assignment mechanism** that solely determine the correct level of clustering.

No clustered sampling:

- If you have a random sample with random treatment assignment, clustering isn't appropriate
- If the sample is a large fraction of the population and treatment effects are heterogeneous across units, robust standard errors are already conservative

Clustered Standard Errors

In short, it's the **sampling process** and the **treatment assignment mechanism** that solely determine the correct level of clustering.

Clustered assignment:

- If assignment is perfectly clustered (all units in same cluster have same treatment assignment), clustering is justified
- If assignment is partially clustered and cluster sizes are large, clustering is justified but standard methods can be extremely conservative

Clustered Standard Errors

In short, it's the **sampling process** and the **treatment assignment mechanism** that solely determine the correct level of clustering.

Cluster Sampling:

- If clusters themselves are sampled, then adjusting for clustering is justified
- If the fraction of sampled clusters is small relative to the total in the population, clustered variance estimator is asymptotically correct (regardless of clustered assignment)
- Same holds if the fraction of sampled clusters is high but only a small fraction of within-cluster observations are sampled
- In other cases, cluster estimates will be overly conservative

Alternative Standard Errors

`feols()` also makes it easy in case we're concerned about alternate correlation structures

- Newey-West (heteroskedastic and serially correlated errors)
  - Panel or Time Series
- Driscoll Kray (cross-sectionally and serially correlated)
  - Panel or Time Series
- Conley (spatially correlated)
- Externally-computed covariance matrices[7]
  - i.e. Abadie et al. 2023 Causal cluster variance (CCV), two-stage cluster bootstrap (TSCB)

[7] The **sandwich** package allows for a wide range of clustered and bootstrapped standard errors.

Conley Standard Errors

If we believe in error dependency based on **spatial proximity**, we can correct for that using **Conley standard errors**.

For example, if we think this dependency likely extends up to 100km, we can specify Conley standard errors as

```
feols(claims ~ wetland_ha, data = wl_df,
      vcov = conley(cutoff = 100))
```

```
## OLS estimation, Dep. Var.: claims
## Observations: 131,588
## Standard-errors: Conley (100km)
##                 Estimate  Std. Error t value Pr(>|t|)
## (Intercept) 101588.31268 47509.19406 2.13829 0.032495 *
## wetland_ha       5.94764     2.93937 2.02344 0.043030 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 3,154,968.2   Adj. R2: 6.545e-5
```

Driscoll Kray Standard Errors

Driscoll Kray standard errors provide an estimate robust to both spatial and temporal dependence.

To estimate Driscoll Kray with a 1 year lag:

```
feols(claims ~ wetland_ha, data = wl_df,
      vcov = DK ~ year)
```

```
## OLS estimation, Dep. Var.: claims
## Observations: 131,588
## Standard-errors: Driscoll-Kraay (L=1)
##                 Estimate  Std. Error t value Pr(>|t|)
## (Intercept) 101588.31268 21118.20440 4.81046 0.017106 *
## wetland_ha       5.94764     1.75838 3.38245 0.043014 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 3,154,968.2   Adj. R2: 6.545e-5
```

Driscoll Kray Standard Errors

Driscoll Kray standard errors provide an estimate robust to both spatial and temporal dependence.

Comparing to a two-year lag:

```
feols(claims ~ wetland_ha, data = wl_df,
      vcov = DK(2) ~ year)
```

```
## OLS estimation, Dep. Var.: claims
## Observations: 131,588
## Standard-errors: Driscoll-Kraay (L=2)
##                 Estimate  Std. Error t value Pr(>|t|)
## (Intercept) 101588.31268 22330.49234 4.54931 0.019898 *
## wetland_ha       5.94764     2.00352 2.96860 0.059135 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 3,154,968.2   Adj. R2: 6.545e-5
```

Newey West Standard Errors

Newey West (1987) provide a heteroskedasticity and autocorrelation consistent covariance estimator.

To estimate Newey West standard errors, we need to specify the panel structure and the desired lag. For a two-year lag:

```
feols(claims ~ wetland_ha, data = wl_df,
      vcov = NW(2) ~ zip + year)
```

```
## OLS estimation, Dep. Var.: claims
## Observations: 131,588
## Standard-errors: Newey-West (L=2)
##                 Estimate  Std. Error t value  Pr(>|t|)
## (Intercept) 101588.31268 10445.26830 9.72577 0.0023089 **
## wetland_ha       5.94764     1.89476 3.13899 0.0517038 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 3,154,968.2   Adj. R2: 6.545e-5
```

Table of Contents

1. Prologue

2. Basic Regression Analysis

3. Fast Fixed Effects Regression

4. (Programmatic) Formulas

5. Standard Errors